

Video = 2D + Time

-> Nhiều ảnh liên tiếp -> Shape: $T \times 3 \times W \times H$

Video không chỉ hỏi “Cái đó là gì?” mà còn hỏi “Nó đang làm gì?”

1. Problem: Video khá là nặng

- SD (640x480): 1.5GB / minute

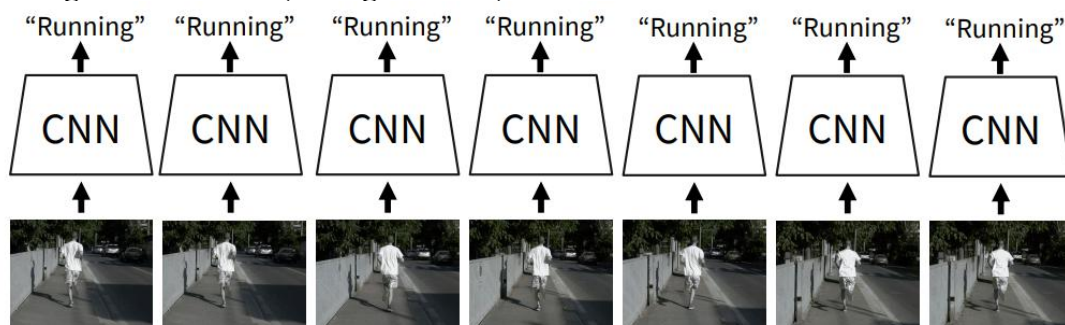
- HD (1920 x 1080): 10GB / minute

FPS trung bình khoảng từ 25 - 30 fps

-> Solution: train trên clip ngắn (ít fps và giảm chiều)

Trong train, model học trên các clip ngắn. Trong test, ta lấy nhiều clip ở các đoạn khác nhau của video rồi average prediction. Điều này giúp mô hình tận dụng nhiều phần của video mà không cần xử lý tất cả một lúc

2. Single-frame CNN (strong baseline)



Cách đơn giản nhất là xem video như tập hợp nhiều ảnh độc lập. Ta chạy một 2D CNN trên từng frame riêng lẻ, sau đó ở test time thì lấy trung bình xác suất dự đoán của các frame

Nhược điểm:

+ Không thật sự hiểu motion giữa các frame. Tuy vậy, trong nhiều dataset, background và pose đã đủ gợi ý nhận nên baseline này vẫn mạnh

+ Dễ nhầm các action có appearance giống nhau nhưng motion khác nhau

-> Nên sử dụng cái này trước

Một chủ đề khác: làm sao để chọn được các frame cần train (có nhiều frame giống nhau, nhiều frame đang là chuyển cảnh) -> một bài toán mới?

3. Late Fusion

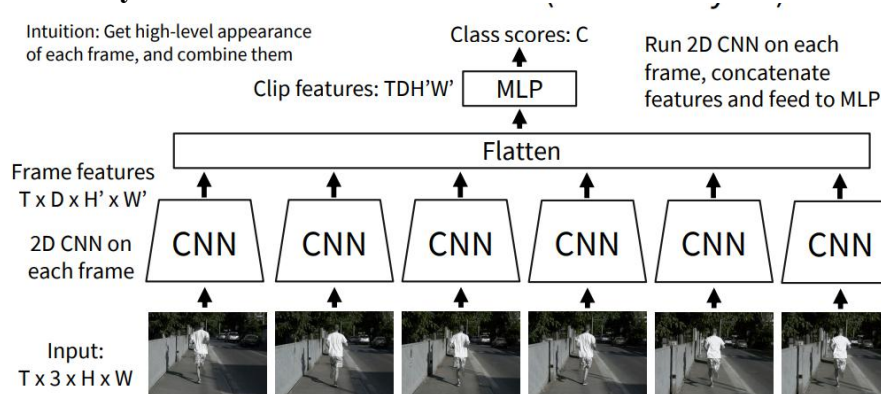
Ý tưởng:

- Chạy 2D CNN trên từng frame (giống single-frame CNN)

- Lấy feature từng frame

- Gộp các feature này lại

3.1. With FC layer



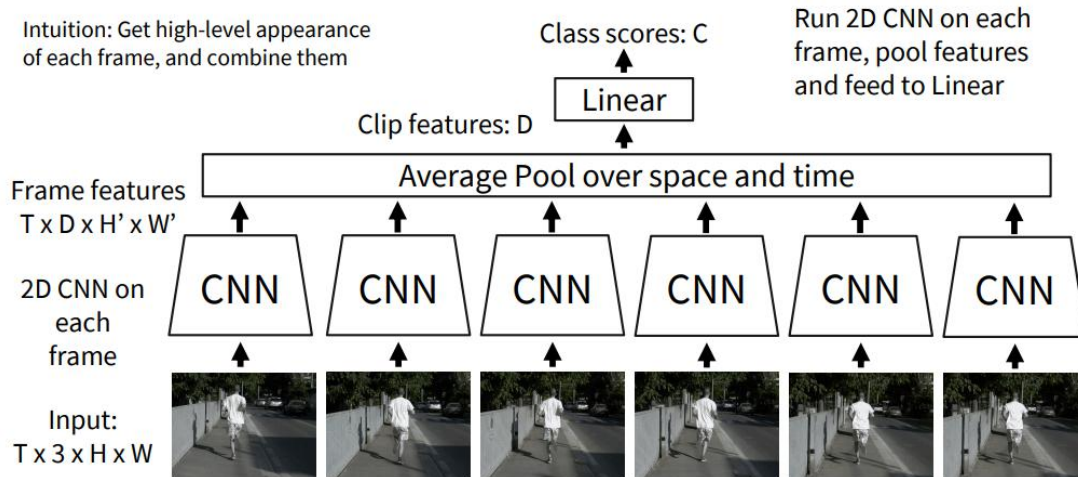
Feature từ tất cả frame được concatenate lại và flatten, rồi cho qua MLP để ra class score

MLP là fully-connected dùng để học cách kết hợp thông tin giữa các frame và cho ra kết quả class score

Nhược: nhiều parameter vì là FC nếu càng nhiều T là càng dài

3.2. With pooling

Thay vì dùng flatten như 3.1 thì ở đây người ta nén lại thành 1 vector nhỏ hơn (1 chiều) rồi sau đó đưa vào linear thôi



Tuy nhiên nó sẽ không lấy được nhiều thông tin như với FC

Nhược điểm chung của late fusion: vì nó kết hợp ở late nên có thể sẽ loss 1 vài thông tin trước đó và vì mỗi frame được xử lý riêng biệt trước nên model khó học được các thay đổi nhỏ giữa các frame liên tiếp (vd nó sẽ khó phân biệt giữa vẫy tay và giờ tay)

4. Early Fusion

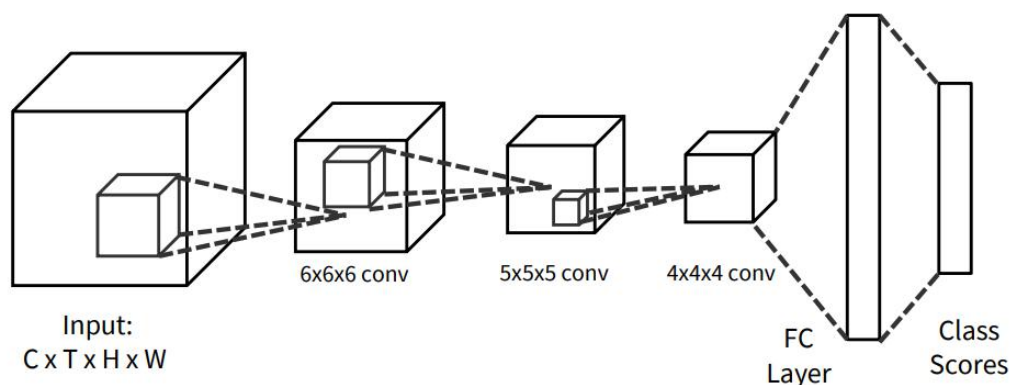
Ở Early Fusion, ta lấy input $T \times 3 \times H \times W$ rồi reshape thành $3T \times H \times W$, tức là chồng tất cả frame theo chiều channel, sau đó đưa vào một 2D CNN bình thường. Ý tưởng là lớp conv đầu tiên sẽ nhìn đồng thời nhiều frame và học so sánh giữa chúng. Nhược điểm: Sau lớp conv đầu tiên, thông tin thời gian bị nén mất. Toàn bộ network phía sau lại chỉ là CNN 2D bình thường -> tức là temporal processing xảy ra đúng 1 lần -> không đủ cho motion phức tạp

5. 3D CNN: convolution trong cả không gian lẫn thời gian

Thay vì conv 2D chỉ trượt theo H, W, ta dùng 3D convolution trượt theo cả T, H, W.

Input lúc này là $3 \times T \times H \times W$, và mỗi layer giữ cấu trúc $D \times T \times H \times W$

3D CNN fuse temporal information dần dần qua nhiều layer, không quá sớm như early fusion, không quá muộn như late fusion -> slow fusion



6. C3D: The VGG of 3D CNNs

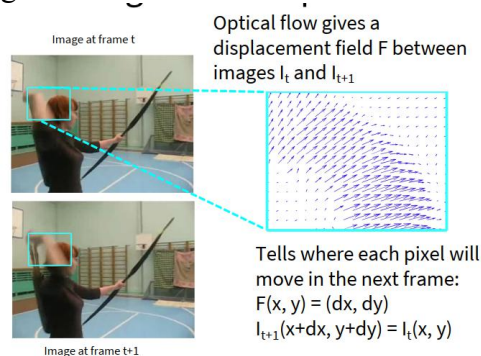
3D CNN sử dụng toàn 3x3x3 conv và 2x2x2 pooling (ngoại trừ Pool1 là 1x2x2 -> giữ temporal resolution ở đầu mạng, tránh mất motion info quá nhanh)

Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

Cái này thường được mọi người dùng để extract feature từ video
Nhược điểm conv3x3x3 rất tốn parameter

7. Optical Flow

Là trường dịch chuyển giữa frame t và t+1



Ý tưởng là điểm ảnh đang ở vị trí (x,y) trong frame t sẽ xuất hiện ở vị trí mới (x+dx,y+dy) trong frame t+1 -> pixel này trôi đi đâu

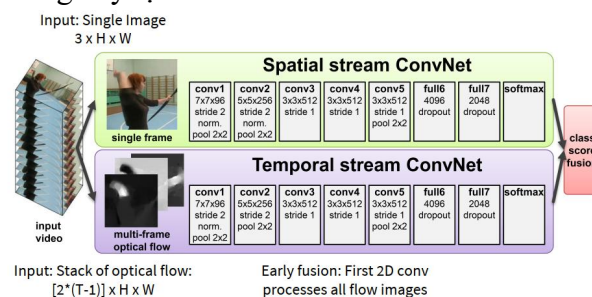
Ưu điểm của cái này là tách được motion ra khỏi appearance

-> Ra ý tưởng chạy 2 luồng: Spatial và Temporal

+ Spatial là ảnh RGB giúp học appearance

+ Temporal là stack optical flow giúp học motion

Xong rồi kết hợp 2 luồng này lại



8. CNN + RNN

Đối với các bài toán có hành động dài tầm 2 tới 5 giây ta khó có thể xài 3D CNN

-> ta kết hợp giữa CNN và RNN -> Recurrent Convolutional Network

Feature ở layer L, thời điểm t phụ thuộc vào:

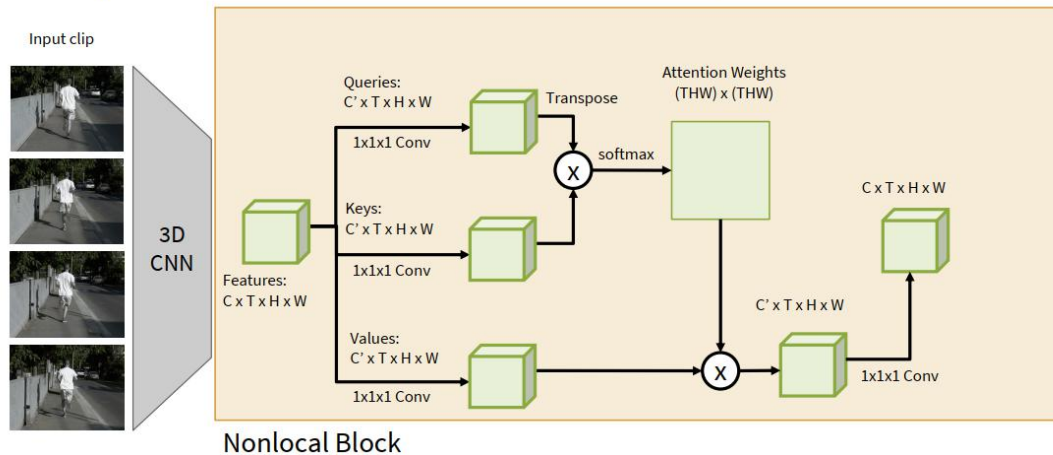
+ feature từ layer L-1 tại cùng thời điểm t

+ feature từ layer L ở thời điểm trước t-1.

Tức là vẫn giữ cấu trúc không gian (giống RNN thay bằng 2D Conv)

Nhược điểm: Nó vẫn chậm như RNN bình thường

9. Self Attention (Non-local Block)



Khá là giống với self attention của 2D chỉ khác nhau là đầu vào của input và ở chỗ mỗi vị trí không chỉ attend theo không gian, mà còn attend qua các frame khác nhau

Để tối ưu ta xài đan xen 3DCNN -> Nonlocal Block -> 3DCNN -> NonlocalBlock -> 3D CNN

10. Mở rộng: Video Transformer

Là mô hình dùng transformer làm backbone chính cho video, video được chia thành tokens, tokens tương tác với nhau qua self-attention

Có 3 hướng chính trong Video Transformer:

- + Factorized attention: Tách attention theo time và space
- + Pooling module: giảm số token trong quá trình mạng chạy
- + Video Masked Autoencoder: che nhiều patch -> model khôi phục

